

Management Report



1. Résumé Exécutif

L'application Ryms est une solution de bureau (Desktop) conçue pour la gestion de tournois, d'équipes et de produits e-sport. Contrairement à une application web classique, elle repose sur une architecture Client Lourd développée en Java, privilégiant la robustesse et une séparation stricte des responsabilités via des modèles de conception (Design Patterns) éprouvés.

L'architecture est modulaire et prépare le terrain pour une maintenance aisée, bien que le choix d'une application de bureau (JavaFX) implique des contraintes de déploiement différentes d'une solution Cloud/Web.

2. Architecture Globale

L'application suit une Architecture en Couches stricte, divisant le code en trois niveaux logiques distincts pour réduire le couplage et faciliter les tests.

Le Modèle en 3 Couches

1. **Couche Présentation (View) :**
 - Gère l'interface utilisateur et les interactions.
 - Technologie : **JavaFX**.
 - Structure : Utilise des fichiers **.fxml** pour le design visuel et des classes **Controller** pour la logique d'affichage (ex: **MatchListViewController**, **LoginController**).
2. **Couche Métier (Business Logic) :**
 - Contient les règles de gestion (tournois, paniers, utilisateurs).
 - Structure : Organisée en "Managers" (**TournamentManager**, **BasketManager**) et exposée via des **Façades** (**SessionFacade**, **GameCatalogFacade**).
3. **Couche Persistance (Persist Logic) :**
 - Gère l'accès aux données.
 - Structure : Utilise le pattern **DAO (Data Access Object)**. L'application définit des interfaces abstraites (ex: **UserDAO**) et leurs implémentations concrètes pour PostgreSQL (**UserDAOPostgres**).

3. Stack Technologique

Composant	Technologie	Observations / Justification
Langage	Java	Langage robuste, typage fort, standard industriel.
Interface Graphique	JavaFX	Framework moderne pour applications de bureau riches. Utilisation de FXML pour séparer le design du code ⁵ .
Base de Données	PostgreSQL 17	SGBD relationnel puissant. Déployé via Docker (postgres:17) ⁶ .
Infrastructure	Docker	Conteneurisation de la base de données via docker-compose.yml , garantissant un environnement de développement iso-prod ⁷ .
Build Tool	Maven	Gestion des dépendances et du cycle de vie du projet (fichier pom.xml présent) ⁸⁸ . +1

4. Analyse des Choix Architecturaux Clés

4.1. Pattern "Abstract Factory" (Flexibilité)

L'application utilise le pattern **Abstract Factory** (classe `AbsFactory` et `PostgresFactory`).

- **Avantage stratégique** : Cela permet de changer de système de base de données (par exemple passer de PostgreSQL à MySQL ou Oracle) sans modifier une seule ligne de la logique métier. Le code métier ne connaît que les interfaces (`UserDAO`), pas les implémentations SQL.

4.2. Pattern "Facade" (Simplicité)

Des classes comme `SessionFacade` ou `TournamentFacade` sont utilisées.

- **Avantage stratégique** : Elles offrent une porte d'entrée unique et simplifiée pour l'interface graphique. Le contrôleur de vue n'a pas besoin de savoir qu'une action implique trois gestionnaires différents; il appelle simplement la Façade.

4.3. Modèle de Données (PostgreSQL)

Le schéma de base de données (`init.sql`) est relationnel et normalisé :

- Tables principales : `users`, `teams`, `games`, `tournaments`, `products`.
- Sécurité : Les clés étrangères (`FOREIGN KEY`) avec suppression en cascade (`ON DELETE CASCADE`) sont correctement configurées pour maintenir l'intégrité des données (ex: supprimer un tournoi supprime ses matchs).

5. Points d'Attention et Risques

5.1. Sécurité des Données

- **Mot de passe** : Le fichier d'initialisation SQL montre des mots de passe en clair (`password_123`) pour les données de test.
 - *Recommandation* : S'assurer que le code de production (`UserManager`) utilise un hachage fort (ex: BCrypt) avant l'insertion en base, ce qui semble être prévu via les exceptions gérées (`IncorrectPasswordException`).

5.2. Gestion de la Connexion (Singleton)

L'application semble utiliser une classe `DBConfig` ou similaire pour la connexion.

- **Risque** : Si la gestion des connexions n'utilise pas de "Pool de connexions" (comme HikariCP), les performances pourraient se dégrader avec un grand nombre d'accès simultanés à la base.

5.3. Déploiement Client Lourd

- Comme c'est une application JavaFX, chaque utilisateur doit l'installer sur sa machine.

- *Recommandation* : Prévoir une stratégie de mise à jour automatique ou un installeur packagé (via `jpackage`) pour faciliter la distribution.

6. Conclusion

L'architecture de **Ryms** est **saine, structurée et professionnelle**. L'équipe de développement a fait le choix de ne pas coupler fortement l'application à la base de données (via le pattern Factory/DAO), ce qui est un gage de qualité et de pérennité. L'utilisation de Docker pour la base de données démontre également une approche moderne des environnements de développement.

Validation : L'architecture est validée pour une mise en production, sous réserve d'un audit de sécurité sur la gestion des mots de passe et d'une validation des performances de la couche graphique JavaFX.